

Bilan S01

Parcours SQL/Data — Semaine 01 : PostgreSQL, SQL et Git

Période : J01 à J07 — 14 au 20 septembre 2026

Statut : Semaine 01 validée

Objectif de la semaine

Établir un environnement PostgreSQL professionnel sous Windows 11, comprendre son organisation logique, créer une première structure de données, pratiquer DDL/DML/SELECT et transactions, puis documenter et versionner le travail avec Git/GitHub.

Bilan par journée

J01 — Audit PostgreSQL

- PostgreSQL/psql 17.5 vérifié.
- Service Windows et processus PostgreSQL contrôlés.
- Connexion avec psql et audit de l'environnement.
- Bases, rôles, schémas, tables, version, port et configuration examinés.
- Hiérarchie serveur → base → schéma → table comprise.

J02 — Base et schémas

- Base sql_training utilisée comme environnement isolé.
- Schémas raw, staging, analytics, sandbox.
- Rôle fonctionnel des différentes couches compris.
- Qualification des objets SQL consolidée.

J03 — Git/GitHub

- Dépôt local initialisé et documenté.
- Branche main.
- Remote origin.
- Commits et premier push vers GitHub.
- git status, git log, git branch -vv et contrôles du remote pratiqués.

J04 — DDL PostgreSQL

- Table raw.training_events créée.
- Types, PRIMARY KEY, NOT NULL, DEFAULT et IDENTITY.
- Métadonnées et contraintes interrogées.
- Première erreur de contrainte analysée.

J05 — DML et transactions

- INSERT, SELECT, UPDATE, DELETE.
- Filtres, alias, expressions et tris.
- NULL, IS NULL, IS NOT NULL.
- Transaction avec BEGIN / ROLLBACK.

J06 — Consolidation autonome

- Challenges SQL réussis.
- Transaction contrôlée.

- Séquence IDENTITY identifiée et interrogée.
- Différence entre nombre de lignes, MAX(event_id) et valeur de séquence comprise.
- Script SQL reproductible validé.

J07 — Revue hebdomadaire

- Concepts expliqués sans simple reproduction.
- Challenge multicritère corrigé et validé.
- Métadonnées via information_schema.
- UPDATE temporaire puis ROLLBACK.
- Script hebdomadaire exécuté sans erreur.

Acquis techniques S01

PostgreSQL

L'utilisateur sait se connecter, auditer l'environnement, naviguer entre base/schéma/table, consulter les métadonnées et raisonner sur une colonne IDENTITY et sa séquence.

SQL

Les fondations sont opérationnelles : DDL, DML, SELECT, WHERE, comparaisons, AND/OR, NULL, ORDER BY, expressions, transactions et contrôles de résultats.

Git/GitHub

Le workflow est opérationnel :

Working Directory → Staging Area → Local Repository → origin/main.

La différence entre git status (état courant) et git log (historique) est consolidée.

Erreurs transformées en apprentissages

- Avertissement Windows code page 850 / 1252 : identifié comme non bloquant.
- Violation NOT NULL : compréhension concrète d'une contrainte.
- Gaps IDENTITY : une valeur consommée n'implique pas une ligne persistante.
- ROLLBACK et séquence : la transaction restaure les données, mais une séquence peut continuer d'avancer.
- Priorité AND sur OR : nécessité du parenthésage pour traduire la logique métier.
- Syntaxe Git -5 : distinction entre argument de révision et option de limitation.

KPI S01

Indicateur	Résultat	
---	---	
Journées réalisées	7 / 7	
Environnement PostgreSQL	Opérationnel	
Base de formation	sql_training	
Schémas métier créés	4 : raw, staging, analytics, sandbox	
Première table RAW	raw.training_events	
Lignes persistantes finales	7	
DDL	Acquis fondamentaux	
DML	Acquis fondamentaux	
SELECT / filtres / tris	Acquis fondamentaux	
NULL	Compris et démontré	
Transactions	Compris et démontré	
Métadonnées PostgreSQL	Interrogées avec succès	
IDENTITY / séquence	Compréhension consolidée	

Git local + GitHub	Opérationnel
Scripts J06/J07	Reproductibles et validés
Documentation quotidienne	J01 à J07
Challenge J07	Validé

Évaluation qualitative

Fondations S01 acquises, sans lacune bloquante identifiée pour passer à S02.

L'acquis majeur n'est pas seulement syntaxique : l'utilisateur commence à raisonner sur l'état des données, les contraintes, les transactions, les métadonnées et la traçabilité du travail.

DERS — état après S01

Le DERS est un indicateur de progression sur l'ensemble des 16 semaines, et non une note à maximiser dès S01.

- SQL (20 pts) : fondamentaux engagés et opérationnels.
- PostgreSQL (10 pts) : environnement, objets, contraintes, métadonnées et transactions engagés.
- Git (5 pts) : workflow local/distant opérationnel.
- Data Modeling, Analytics Engineering, Python/Data Engineering, Power BI, Cloud/Orchestration, Business/Product : non encore suffisamment couverts pour une notation significative.

Le suivi chiffré complet du DERS deviendra plus pertinent aux Gates prévues dans le parcours.

Capital pédagogique créé

La semaine peut être réutilisée pour produire :

1. « Auditer PostgreSQL sous Windows 11 avec PowerShell et psql ».
2. « Serveur, base, schéma et table : comprendre PostgreSQL ».
3. « Pourquoi NULL = NULL ne fonctionne pas comme une comparaison classique ».
4. « Sécuriser un UPDATE avec BEGIN / ROLLBACK ».
5. « Comprendre les gaps IDENTITY dans PostgreSQL ».
6. « Git : Working Directory → Staging → Commit → Push ».
7. Un mini-lab SQL basé sur raw.training_events.

Points de vigilance pour S02

- Continuer à écrire les requêtes avant de regarder une solution.
- Maintenir la qualification schema.table.
- Continuer les contrôles avant/après les opérations DML.
- Documenter les erreurs, pas seulement les réussites.
- Réduire progressivement le dépassement du temps prévu sans sacrifier la documentation.
- Maintenir le cycle : Apprendre → Exécuter → Tester → Capturer → Expliquer → Documenter → Versionner → Capitaliser.

Décision de passage

S01 validée → passage à S02.

S02 pourra approfondir les fondamentaux SQL sur un jeu de données métier et augmenter progressivement l'autonomie, conformément au Plan d'exécution SQL/Data de 16 semaines.